

Day 8 - OOP Fundamentals: Classes, Constructors, Encapsulation

Day 8 — OOP Fundamentals: Classes, Constructors, Encapsulation

Objectives

- Understand what a class is in JavaScript and how `class` syntax works
- Understand the constructor and how `this` behaves inside a class (now that you understand `this` in general from Day 4)
- Understand private fields (`#field`) and why encapsulation — hiding internal details — is useful
- Understand instance methods vs. data, and when a class is the right tool

A class is a blueprint; an instance is a real object built from it

Exactly the same core idea as any object-oriented language: a **class** is a blueprint describing what every object built from it will have; an **instance** is one actual object built from that blueprint.

```
class Dog {
  constructor(name, breed) {
    this.name = name;
    this.breed = breed;
  }

  bark() {
    return `${this.name} says Woof!`;
  }
}

const rex = new Dog("Rex", "Labrador");    // `new` creates a brand-new INSTANCE of the Dog class
const fido = new Dog("Fido", "Poodle");

console.log(rex.bark());    // Rex says Woof!
console.log(fido.bark());    // Fido says Woof!
```

The `new` keyword is required in JavaScript when creating an instance of a class — forgetting it (`Dog("Rex", "Labrador")` without `new`) causes an error, since `Dog` is specifically designed to be used as a class, not called as a plain function.

The constructor — JavaScript's version of Day 8's `__init__` (if you know Python) or just "setup logic"

constructor is a specially-named method that runs automatically the moment you create a new instance with **new**. Its job is to set up that instance's starting data:

```
class Point {
  constructor(x, y) {
    this.x = x;
    this.y = y;
  }
}

const p = new Point(3, 4);
console.log(p.x, p.y); // 3 4
```

Inside the constructor (and inside every regular method on a class), **this** refers to the specific instance being created or operated on — exactly the "called as a method" rule from Day 4's **this** explanation, since **new Dog(...)/rex.bark()** are both, under the hood, calling their functions in a way that correctly sets **this**.

Methods

A method is simply a function defined directly inside the class body (notice: no **function** keyword needed, unlike a plain object's methods from Day 4):

```
class Counter {
  constructor() {
    this.count = 0;
  }

  increment() {
    this.count += 1;
  }

  reset() {
    this.count = 0;
  }
}

const c1 = new Counter();
const c2 = new Counter();
c1.increment();
c1.increment();
console.log(c1.count); // 2
console.log(c2.count); // 0 -- completely separate from c1
```

Each instance gets its own independent copy of whatever's set in the constructor (**this.count** here) — **c1** and **c2** never interfere with each other, exactly like the Python track's equivalent example, if you've seen it.

Private fields — hiding internal details with `#`

Sometimes you want a piece of data to belong to an instance, but to be genuinely inaccessible from *outside* the class entirely — not just "not intended to be touched," but actually blocked by the language itself.

JavaScript supports this with a **#** prefix:

```
class BankAccount {
  #balance; // declares a PRIVATE field -- only accessible from inside this class
```

```

    constructor(owner, initialBalance) {
        this.owner = owner;
        this.#balance = initialBalance;
    }

    deposit(amount) {
        this.#balance += amount;
    }

    getBalance() {
        return this.#balance;
    }
}

const account = new BankAccount("Ana", 100);
account.deposit(50);
console.log(account.getBalance());    // 150
console.log(account.#balance);        // SyntaxError -- #balance is not accessible from outside t

```

This idea — deliberately hiding a piece of data and only exposing controlled ways to read or change it (here, `getBalance()` and `deposit()`) — is called **encapsulation**. Why bother hiding `#balance` at all, instead of just using `this.balance` (no `#`, freely accessible)? Because it lets the class guarantee its own rules: for instance, you could add validation inside `deposit()` (reject negative amounts) with total confidence that `#balance` can *only* ever be changed through methods that enforce that rule — nothing outside the class has any way to reach in and set `#balance` directly, bypassing your checks. Without the `#`, any outside code could do `account.balance = -99999` directly, completely ignoring any rules your methods were trying to enforce.

Getters and setters — methods that look like plain property access

You can define special methods that run automatically when a property is *read* or *assigned*, using `get` and `set`, letting you add logic (like validation, or a computed value) while still allowing the natural-looking `account.balance` syntax rather than requiring `account.getBalance()`:

```

class BankAccount {
    #balance;

    constructor(initialBalance) {
        this.#balance = initialBalance;
    }

    get balance() {
        return this.#balance;
    }

    set balance(amount) {
        if (amount < 0) {
            throw new Error("Balance cannot be negative");
        }
        this.#balance = amount;
    }
}

const account = new BankAccount(100);

```

```
console.log(account.balance); // 150 -- reads like a plain property, but actually calls the getter
account.balance = 200;       // looks like plain assignment, but actually calls the setter method
account.balance = -50;       // throws an Error, thanks to the setter's validation
```

This is a nice piece of syntax sugar: callers write `account.balance` and `account.balance = 200` exactly as if `balance` were an ordinary, plain property — but behind the scenes, your `get/set` methods are quietly running, giving you the chance to compute a value or validate an assignment.

When should you actually reach for a class, versus a plain object or a plain function?

Same practical guidance as any object-oriented language: use a **plain function** when you're just computing something from inputs, with no state to remember. Use a **plain object** (Day 3) for a simple bundle of related data, with no enforced guarantees about its shape and no meaningful behavior attached. Use a **class** once you have data *and* behavior that clearly belong together, especially when you want to guarantee every instance has a consistent shape, or you specifically want to hide internal details behind a controlled interface (private fields plus getters/setters, as above).

Exercises

Open `exercises.js`, fill in each `// TODO` inside the class definitions provided, and run `node exercises.js`.